

Module 4

UVM Environment & Checker Maturity (SV/UVM)

Learning objectives

- Design and mature your ****UVM environment**** (env, agents, drivers, monitors, sequencers) and ****check...**

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Environment and Agent Archit"]

T2["Transactions and Sequences"]

T1 --> T2

T3["Drivers and Monitors"]

T2 --> T3

Design and mature your **UVM environment** (env, agents, drivers, monitors, sequencers) and **checking strategy** (scoreboards, reference m...

Overview

- Modules 1–3 focused on planning (scope, tests, regressions) and measurement (coverage). In Module 4...

How to learn this module

Suggested learning path (1/2)

- Re-read ``module2/TEST_PLAN.md`` and ``module3/COVERAGE_DESIGN.md``.
- Scaffold Module 4 workspace: ``. /scripts/module4.sh --scaffold``
- Complete ``module4/ENV_DESIGN.md`` (agents, transactions, connectivity).
- Complete ``module4/CHECKER_PLAN.md`` (scoreboard, reference model, SVA split).
- Implement or refine UVM components in ``module4/tb/`` and ``common_dut/tb/``.

Suggested learning path (2/2)

- Validate: ``./scripts/module4.sh --check``

Design architecture

1. Mature UVM environment (stream_fifo)

- Env (``stream_fifo_env``):
connects source/sink agents,
scoreboard, coverage
subscribers.
- Agents: driver + sequencer +
monitor per interface; active vs
passive roles documented in
``ENV_DESI...`
- Transactions: item fields mirror
DUT handshake semantics (data,
valid/ready timing).
- Reference implementation sketch:
``module4/tb/tb_stream_fifo_uvm.sv`
and `stream_pkg.sv` patterns.`

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
subgraph DUT["stream_fifo (common_dut/rtl)"]
SRC["Source interface\ns_valid / s_ready / s_data"]
MEM["Circular buffer\nmem[DEPTH], wr_ptr, rd_ptr"]
SNK["Sink interface\nm_valid / m_ready / m_data"]
STS["Status\nlevel, overflow, underflow"]
end
```

2. Checking architecture

- Scoreboard / reference model — golden data path and ordering checks (`CHECKER\_PLAN.md`).
- SVA — protocol and functional assertions on interfaces and internal flags.
- Clear split: what is checked in SVA vs scoreboard vs test self-check.

Verification Architecture

```
Diagram: Verification Architecture
(render with mmdc when available)
flowchart TB
    subgraph ENV["stream_fifo UVM env"]
        SEQ["Sequences / transactions"]
        DRV["Drivers"]
        MON["Monitors"]
        SB["Scoreboard / ref model"]
```

3. UVM phase execution flow

- build_phase: create env, agents, scoreboard, coverage subscribers via factory.
- connect_phase: connect monitor analysis ports to scoreboard and coverage; tie virtual interfaces.
- run_phase: raise objection, start sequences on sequencers, wait for drain, drop objection.
- report_phase: summarize errors, assertion failures, and coverage sampling status.
- Top module ``module4/tb/tb_stream_fifo_uvm.sv`` instantiates DUT and calls ``run_test()``.

DUT instantiation in UVM top

```
// DUT instance
stream_fifo #(
    .DATA_WIDTH (DATA_WIDTH),
    .DEPTH      (DEPTH)
) dut (
    .clk        (clk),
    .rst_n      (rst_n),
    // source side
    .s_valid    (s_if.valid),
    .s_data     (s_if.data),
    .s_ready    (s_if.ready),
    // sink side
    .m_valid    (m_if.valid),
    .m_data     (m_if.data),
    .m_ready    (m_if.ready),
    // status
```

```
# ...
```

stream_env — build & connect

```
class stream_env extends uvm_env;

    `uvm_component_utils(stream_env)

    stream_driver      drv;
    stream_monitor     mon;
    stream_scoreboard sb;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        drv = stream_driver      ::type_id::create("drv", this);
        mon = stream_monitor     ::type_id::create("mon", this);
    endfunction
endclass
```

...

Scoreboard write() hook

```
class stream_scoreboard extends uvm_component;

    `uvm_component_utils(stream_scoreboard)

    uvm_analysis_imp #(stream_item, stream_scoreboard) imp_in;

    int unsigned num_seen;

    function new(string name, uvm_component parent);
        super.new(name, parent);
        imp_in = new("imp_in", this);
        num_seen = 0;
    endfunction

    // Called for each transaction seen by the monitor
    function void write(stream_item t);

# ...
```

Key files to study

Open these in the repo

- ``module4/ENV_DESIGN.md`` — env, agent, driver, monitor, sequencer design
- ``module4/CHECKER_PLAN.md`` — scoreboard, reference model, and assertion strategy
- ``module4/tb/stream_pkg.sv`` — reference UVM package (env, agents, scoreboard)
- ``module4/tb/tb_stream_fifo_uvm.sv`` — top-level UVM testbench and DUT hookup
- ``common_dut/rtl/stream_fifo.sv`` — DUT under verification
- ``scripts/module4.sh`` — env/checker doc and TB file checks

Verification & testing methods

1. Stimulus and checking flow

- Sequences → driver → DUT →
monitor → scoreboard compare and
assertion sampling.
- Reset, backpressure, and error-
injection scenarios exercised
per `TEST\_PLAN.md`.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["Sequences → driver"] --> DUT["DUT"]

DUT --> MON["Monitor"]

MON --> SB["Scoreboard compare"]

MON --> SVA["SVA Assertion fire"]

SB --> PASS["Test pass/fail"]

2. Assertion-based verification (ABV)

- Place assertions close to interfaces; use ``assert`/`assume`` policy consistent with your simulator f...
- Document waiver and severity policy for known benign cases.

3. Environment review

- Peer/self review of `ENV\_DESIGN.md` and `CHECKER\_PLAN.md` before expanding regression scope.
- `./scripts/module4.sh --check` validates design docs and TB file presence.

stream_test — run_phase

```
class stream_test extends uvm_test;

    `uvm_component_utils(stream_test)

    stream_env env;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = stream_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);

# ...
```

Syllabus topics

1. Environment and Agent Architecture

- Environment and agent architecture (env vs agents vs tests).
- Layered testbench structure and reuse across tiers and DUT variants.

2. Transactions and Sequences

- Transaction and sequence design aligned with your test plan.
- Mapping test intents to UVM sequences and configuration.

3. Drivers and Monitors

- Driver and monitor behavior, including resets, backpressure, and error handling.

4. Scoreboards and Reference Models

- Scoreboard and reference model design.

5. Assertions and Checking Strategy

- Assertion strategy: what to check with SVA vs scoreboards vs tests.

Command reference highlights

Scaffold and validate Module 4

- `./scripts/module4.sh --scaffold`

Inspect UVM env and scoreboard

- `head -80 module4/tb/stream_pkg.sv`

Hands-on examples

Module 4 self-check

```
$ ./scripts/module4.sh --help
```

Terminal: module4_check

```
[[0;36m===== [[0m
[[0;36mModule 4: UVM Environment & Checker Maturity [[0m
[[0;36m===== [[0m
```

Usage: ./scripts/module4.sh [OPTIONS]

Module 4: UVM Environment & Checker Maturity (SV/UVM)
This script summarizes and checks the planning artifacts for Module 4.

OPTIONS:

--env-status	Show status of env_design.md only
--checker-status	Show status of checker_plan.md only
--checklist-status	Show status of checklist_module4.md only
--summary	Show all statuses (default)
--check	Media/CI self-check (structure only)
--demo	Print example commands from EXAMPLES.md
--scaffold	Copy templates/*.md into module4/ if missing
--help, -h	Show this help message

EXAMPLES:

```
# Full summary
./scripts/module4.sh
```

```
# Focus only on environment design
./scripts/module4.sh --env-status
```

Exercise scaffold

```
./scripts/module4.sh --scaffold
```

Demo: Environment design template

```
$ head -30 module4/templates/ENV_DESIGN.md
```

Terminal: ex01_env_design

UVM Environment & Agent Design (Module 4)

> **Purpose**: Define the detailed UVM environment, agent, transaction, and sequence design for
> **Module**: 4 - UVM Environment & Checker Maturity (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module4/templates/` if nee

1. Context

<!-- TODO: DUT name, key interfaces, prior artifacts. Main verification concerns driving env des

2. Environment Block Diagram

<!-- TODO: Describe or sketch environment hierarchy (uvm_test_top, env, agents, driver, monitor,

3. Component Responsibilities

<!-- TODO: Table of Component, Type, Responsibilities, Notes. -->

4. Transaction Models

<!-- TODO: Main transaction classes, key fields, constraints, helper methods, mapping to DUT. --

5. Sequence Design

5.1 Base Sequences

<!-- TODO: Table of Sequence Class, Purpose/Intent, Uses Transaction(s), Notes. -->

5.2 Feature and Stress Sequences

Demo: Checker plan template

```
$ head -25 module4/templates/CHECKER_PLAN.md
```

Terminal: ex02_checker_plan

```
# Checker, Scoreboard & Assertion Plan (Module 4)
```

```
> **Purpose**: Define the functional checking strategy for the DUT: scoreboards, reference model
```

```
> **Module**: 4 - UVM Environment & Checker Maturity (SV/UVM)
```

```
> **Instructions**: Fill in this document for your design. Copy from `module4/templates/` if nee
```

```
## 1. Context
```

```
<!-- TODO: DUT name, key correctness concerns, related documents. Overall checking philosophy (a
```

```
## 2. Scoreboards and Reference Models
```

```
### 2.1 Scoreboards
```

```
<!-- TODO: Table of Scoreboard ID, Component/Class, Inputs, Purpose, Notes. Matching strategy, d
```

```
### 2.2 Reference Models
```

```
<!-- TODO: Table of Ref Model ID, Implementation, Abstraction Level, Used By, Notes. Inputs, out
```

```
## 3. Assertions and Protocol/Functional Checkers
```

```
### 3.1 Assertion Inventory
```

Demo: Module-local UVM TB

```
$ head -35 module4/tb/tb_stream_fifo_uvm.sv
```

Terminal: ex03_module_tb

```
`timescale 1ns/1ps
```

```
// -----  
// UVM-based testbench for the common stream_fifo DUT  
// -----  
//  
// This top-level testbench:  
// - Instantiates the shared DUT: common_dut/rtl/stream_fifo.sv  
// - Instantiates two stream_if interfaces (source and sink)  
// - Connects interfaces to DUT ports  
// - Puts the virtual interfaces into the UVM config DB  
// - Calls run_test("stream_test") defined in stream_pkg.sv  
//  

```

```
`include "uvm_macros.svh"  
import uvm_pkg::*;
```

```
// Import the UVM components for this bench  
`include "stream_if.sv"  
`include "stream_pkg.sv"
```

```
module tb_stream_fifo_uvm;
```

```
    localparam int DATA_WIDTH = 8;  
    localparam int DEPTH      = 4;
```

```
    // Clock and reset  
    logic clk;  
    logic rst_n;
```

```
    // Source and sink interfaces  
    stream_if #(DATA_WIDTH(DATA_WIDTH)) s_if (.*);  
    stream_if #(DATA_WIDTH(DATA_WIDTH)) m_if (.*);
```

```
    // Status from DUT
```

Demo: Reference env design

```
$ head -20 module4/.solutions/ENV_DESIGN.md
```

Terminal: ex04_solutions

UVM Environment & Agent Design (Module 4)

> **Purpose**: Define the detailed UVM environment, agent, transaction, and sequence design for
> **Module**: 4 – UVM Environment & Checker Maturity (SV/UVM)

1. Context

- **DUT**: `stream\_fifo` — parameterizable streaming FIFO with ready/valid handshakes on source
- **Key Interfaces / Protocols**: Source interface (s_valid, s_ready, s_data); sink interface (m)
- **Prior artifacts**:
 - `module1/VERIFICATION\_PLAN.md` (high-level env & strategy).
 - `module2/TEST\_PLAN.md` (test catalogue and types/tiers).
 - `module3/COVERAGE\_DESIGN.md` (coverage placement and IDs).

Summarize the **main verification concerns** that drive your env design (e.g., throughput, order

- **In-order transfer (R1)**: Scoreboard must compare popped data with expected order (reference
- **Overflow/underflow flags (R2, R3)**: Assertions and scoreboard must verify flags set only wh
- **Backpressure and protocol**: Driver and monitor must respect valid/ready handshake; tests mu

Demo: Module validation

```
$ ./scripts/module4.sh --check
```

Terminal: ex05_validate

```
[[0;36m=====[[0m
[[0;36mModule 4: UVM Environment & Checker Maturity[[0m
[[0;36m=====[[0m
```

Module 4: media self-check
OK: module4/ exists
OK: templates/ exists
OK: EXAMPLES.md exists
OK: docs/MODULE4.md exists
OK: common_dut/rtl/ exists

All required checks passed.

Practice & assessment

What you should know (1/2)

- Describe and justify your UVM environment and agent architecture.
- Implement and maintain drivers, monitors, sequencers, and transactions that match your DUT and test...
- Implement or integrate scoreboards and reference models for robust checking.
- Design and place protocol and functional assertions that support your verification goals.
- Configure your environment for different modes and regression tiers.
- UVM env/agent architecture documented in ``module4/ENV_DESIGN.md``.

What you should know (2/2)

- Transaction and sequence designs documented and aligned with `module2/TEST\_PLAN.md`.
- Scoreboards and reference models specified in `module4/CHECKER\_PLAN.md`.
- Key protocol/functional assertions listed and placed.
- At least one env/checker review (self or peer) completed; action items captured.

Assessment checklist

- UVM env/agent architecture documented in ``module4/ENV_DESIGN.md``.
- Transaction and sequence designs documented and aligned with ``module2/TEST_PLAN.md``.
- Scoreboards and reference models specified in ``module4/CHECKER_PLAN.md``.
- Key protocol/functional assertions listed and placed.
- At least one env/checker review (self or peer) completed; action items captured.

Summary & next steps

- Design and mature your ****UVM environment**** (env, agents, drivers, monitors, sequencers) and ****check...**
- Complete module4/CHECKLIST.md
- Review module4/EXAMPLES.md and run each lab
- Next: Next module in course